



VO Computernetzwerke – Kapitel 5

SS 2026

Armin Veichtlbauer

5. Datagramme

Überblick

UDP:

Wie funktioniert UDP? Was ist ein UDP Datagramm? Welche Headerfelder hat UDP und wozu werden sie verwendet? Was ist ein Port und welche Ports sind gängig?

QUIC:

Was ist QUIC? Wie funktioniert QUIC in Verbindung mit UDP? Wie arbeitet QUI mit TLS zusammen? Was kann damit typischerweise gemacht werden?

ICMP:

Was ist ICMP und wie funktioniert es? Welche ICMP Requests gibt es und wozu werden diese verwendet? Auf welcher Schicht arbeitet ICMP?

5.1 Grundlagen der Transportschicht

Grundaufgabe der Transportschicht

- Die Transportschicht stellt den höheren Schichten eine End-to-end Übertragung ihrer Nutzdaten bereit (unter Nutzung der unteren Schichten, also von IP)
 - Verbindungsorientiert: Verwaltung einer virtuellen Verbindung → TCP
 - Verbindungslos: Übertragung einzelner Datagramme → UDP
- Dazu gehören die folgenden Teilaufgaben
 - Binden der Daten an eine Anwendung → Ports
 - Zerlegung der Daten in handhabbare Teile (Segmente oder Datagramme)
 - Weiterleitung der Daten und deren Qualitätsanforderungen an Netzschicht: Fehlertoleranz, Zeitbedingungen (Latenzen) → Quality of Service

5.1 Grundlagen der Transportschicht

Portnummern

- Ports (16 Bit lange Adressen) werden definiert, um eine Zuordnung der Nutzdaten zu Prozessen (Anwendungen) in den Endsystemen ermöglichen
- Beim Request unterscheidet man Sender und Empfänger
 - Source Port: Spezifiziert den Sendeprozess; ist beim Request frei wählbar und wird vom Betriebssystem zugeteilt
 - Destination Port: Spezifiziert den Empfangsprozess; ist beim Request spezifiziert (Dienst „horcht“ unter dieser Portnummer → oft Well-known Ports)
- Beim Response werden die Portnummern getauscht (Source ↔ Destination)

5.1 Grundlagen der Transportschicht

Bedeutende Well-known Ports

Transportprotokolle	Ports	Anwendung
TCP	21	File Transfer Protocol (FTP) – Steuerungsverbindung
TCP	22	Secure Shell (SSH)
TCP, UDP	53	Domain Name System (DNS)
TCP	443	Hypertext Transfer Protocol Secure Version 2 (HTTPS/2)
UDP, QUIC	443	Hypertext Transfer Protocol Secure Version 3 (HTTPS/3)
TCP	465	Simple Mail Transfer Protocol Secure (SMTPS)
TCP	993	Internet Message Access Protocol Secure (IMAPS)

5.1 Grundlagen der Transportschicht

Transportprotokolle

- Transmission Control Protocol (TCP): Verbindungsorientiert und gesichert
- User Datagram Protocol (UDP): Verbindungslos und ungesichert (also ohne Bestätigung); Integritätsüberprüfung über Prüfsumme ist jedoch möglich
- Stream Control Transmission Protocol (SCTP): Sicherung wie bei TCP, aber ein effizientes Zeitverhalten wie bei UDP durch Verwaltung mehrerer Streams
- Quick UDP Internet Connections (QUIC): Ähnlich wie SCTP, baut auf UDP auf
- Transport Layer Security (TLS): Verschlüsselung der Layer 4 Nutzdaten

5.1 Grundlagen der Transportschicht

Sicherung in der Transportschicht

- Fehlersicherung soll nicht reparierte Fehler der unteren Schichten beheben

Schicht	Fehlerursache	Maßnahmen
Transport	Nicht reparierte Fehler	End-to-end ARQ (meist mit Schiebefensterverfahren)
Netzwerk	Pufferüberlauf	Load Balancing (z.B. durch stochastisches Routing)
Leitungssteuerung	Bitfehler, Kollisionen	Zugriffssteuerung, Fehlererkennung, FEC, ARQ
Bitübertragung	Signalfehler	Signalcodierung (z.B. Erhöhung der SNR), Modulation

- Angriffssicherung soll Vertraulichkeit, Integrität, Verfügbarkeit, aber auch Authentizität der übertragenen Daten gewährleisten

5.1 Grundlagen der Transportschicht

Laststeuerung in der Transportschicht

- Flow Control dient der Drosselung des Verkehrs zur Vermeidung von Überlastung des Empfängers
- Congestion Control dient der Drosselung des Verkehrs zur Vermeidung von Überlastung des Netzes
- Realisiert wird das in der Transportschicht über eine Limitierung der Daten, die von den Endsystemen versendet werden dürfen → Bei TCP mit Sliding Window
- Problem: Aktuell steigt der Anteil von UDP Daten im Internet kontinuierlich an; UDP kann das aber nicht limitieren → **Andere Schichten** müssen übernehmen

5.2 User Datagram Protocol

Eigenschaften

- Definiert in RFC 768; Datagramme sind im Grunde nur IP Pakete plus Port-Infos
- Bietet verbindungslose und unzuverlässige Übertragung von Datagrammen zwischen Prozessen auf (nicht notwendigerweise) unterschiedlichen Systemen
- Vorteile und Nachteile
 - Minimaler Overhead → Guter Durchsatz; Eignung für Echtzeitanwendungen
 - Mehr Kontrolle (wann was senden?), aber auch mehr Aufwand für Prozesse (z.B. sind Broad- und Multicasts möglich; in TCP hingegen nicht unterstützt)
 - Keine Wiederholung im Fehlerfall; Reihenfolge der Paketankünfte undefiniert

5.2 User Datagram Protocol

Struktur des UDP Headers

0 - 3	4 - 7	8 - 11	12 - 15	16 - 19	20 - 23	24 - 27	28 - 31
Source Port				Destination Port			
Length				Checksum			

- Source Port: Portnummer des sendenden Prozesses – wird für Antworten benötigt
- Destination Port: Portnummer des empfangenden Prozesses – wird dafür benötigt, um beim Empfänger Endsystem eine Zuordnung zum gewünschten Prozess machen zu können
- Length: Länge des Datagrammes in Bytes (Header + Payload!) – mindestens 8 Bytes
- Checksum: Prüfsumme über das gesamte Datagramm (plus Pseudoheader) zur Integritätsprüfung

5.2 User Datagram Protocol

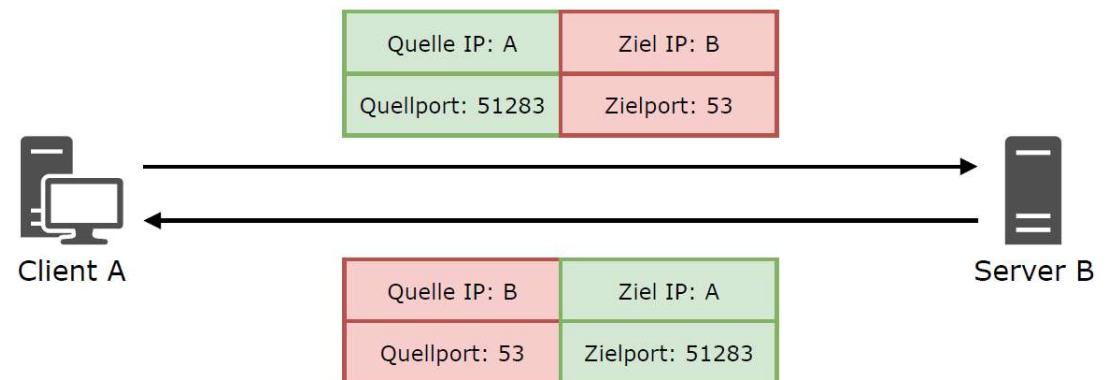
Prüfsummenberechnung

- Prüfsumme ist einziges Sicherungsverfahren, das UDP anbietet: **Integrität** der Daten kann damit gecheckt werden
 - Für Fehlersicherheit / Reliability: Übertragungsfehler können erkannt werden
 - Für die Angriffssicherheit / Security: Manipulation kann aufgedeckt werden
- Verwendung und Auswertung der Prüfsumme sind optional, aber üblich – bei fehlerhafter Prüfsumme wird das Datagramm verworfen
- Prüfsumme wird über gesamtes Datagramm plus so genannten Pseudoheader (beinhaltet die IP Adressen) gebildet

5.2 User Datagram Protocol

Verwendung der Netzwerk Adressen

- Im Request Paket
 - Source Port: 51283
 - Destination Port: 53 (DNS)
- Im Response Paket
 - Source Port: 53 (DNS)
 - Destination Port: 51283
- Source Ports im Client werden vom Betriebssystem vergeben (> 49151)



**Netzwerk Adressen von Client und Server
in Request und Response Paketen**

5.3 Quick UDP Internet Connections

Anforderungen

- Die Netzerkanwendung (der anfragende Prozess) wählt das zu verwendende Transportprotokoll auf Basis der Anforderungen der Netzerkanwendung
- Anforderung Zuverlässigkeit (Reliability): Sicherstellung, dass Daten vollständig und korrekt beim Empfänger ankommen → Transmission Control Protocol (TCP)
- Anforderung Latenzminimierung (Latency): Minimierung von Verzögerungen (Delays) in der Kommunikation → User Datagram Protocol (UDP)
- Im World Wide Web: Je nach ausgetauschten Daten sowohl Reliability als auch Latency Anforderungen möglich → Quick UDP Internet Connections (QUIC)

5.3 Quick UDP Internet Connections

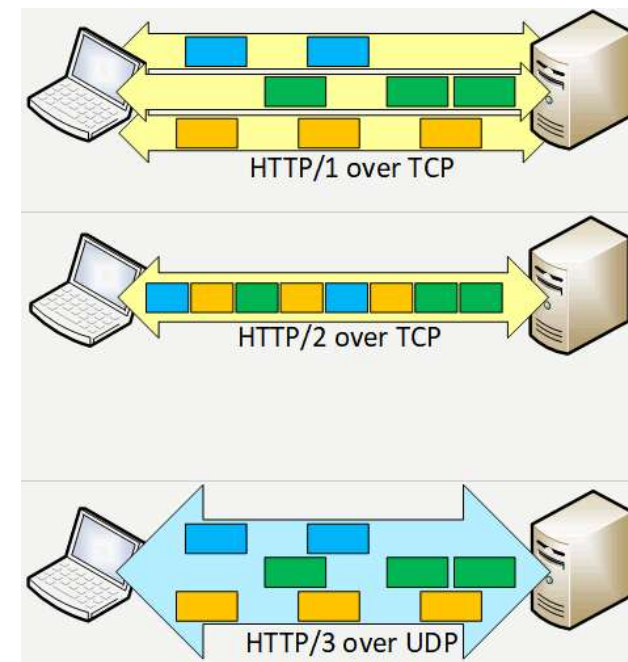
Vorteile von QUIC

- Leichte Implementier- und Adaptierbarkeit: Nutzung von UDP; QUIC selber läuft im User Space als Teil der Anwendung
- Niedrige Latenzen im Verbindungsaufbau: Kombination von 3-Wege-Handshake und TLS Handshake (Built-In-Security mit TLS 1.3)
- Multi Streaming: Unabhängige QUIC Streams (z.B. mehrere HTTP Streams) über eine QUIC Verbindung) mit Forward Error Correction
- Connection Migration: Tracking über 64 bit QUIC Connection ID anstatt der Portnummer (z.B. bei Migration vom Mobilfunknetz ins WLAN)

5.3 Quick UDP Internet Connections

QUIC Streams

- In HTTP/1 wurden pro Datenstrom extra TCP Verbindungen aufgebaut → Keine Blockierung, aber viel Session Management
- In HTTP/2 wurde nur eine TCP Verbindung aufgebaut → Weniger Session Management, aber Fehler blockierten gesamte Verbindung
- QUIC baut eine Multi Stream Verbindung auf
 - Keine Blockierungen zwischen Streams
 - Trotzdem geringes Session Management

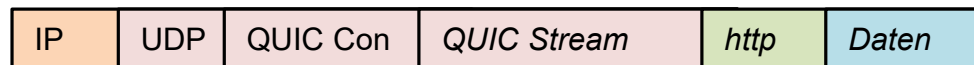


QUIC Streams in HTTP/3

5.3 Quick UDP Internet Connections

Nachteile von QUIC

- Fehlerbehebung kann ebenso aufwändig sein wie TCP: Sequenznummern erlauben Re-ordering per Stream, und Re-transmissions bei Datenverlust
- Verkehrssteuerung im Corebereich des Internet ist schwieriger aufgrund der Verschlüsselung (End-to-end funktionieren aber Flow und Congestion Control)
- Security kann ein Problem sein: Firewalls haben durch Verschlüsselung weniger Zugriff; Wiederverwendung alter Verbindungen ohne erneuten Handshake („0-RTT“) ist riskant

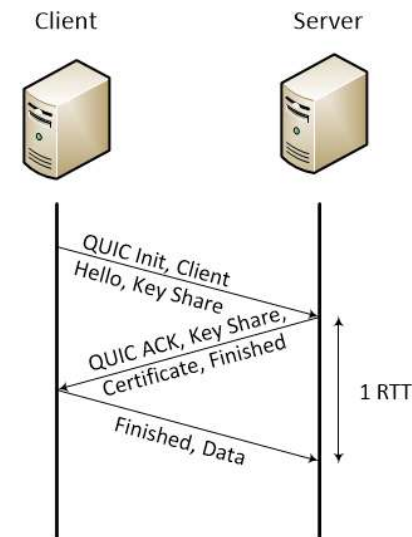
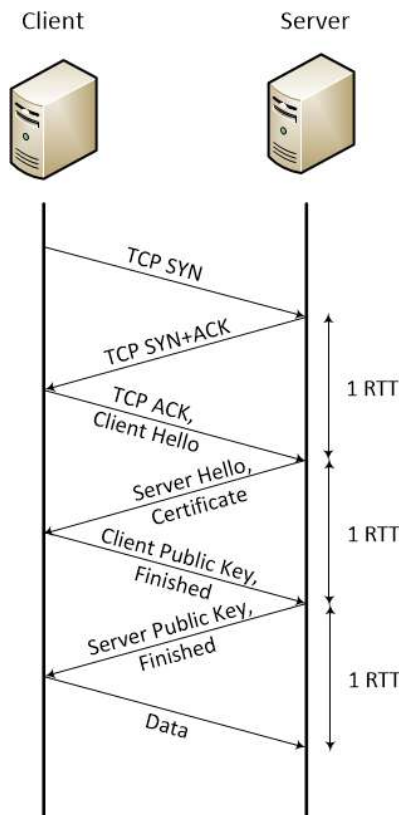


QUIC Paketaufbau

5.3 Quick UDP Internet Connections

QUIC Handshake

- Round Trip Time (RTT) ist die Zeit zwischen Request und Response
- RTT kann im Bereich von 100 ms liegen
- TCP mit TLS 1.3 liegt dazwischen (2 RTTs)
 - TCP Handshake
 - TLS Handshake

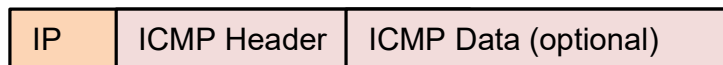


**Handshakes von
TCP / TLS1.2 (links)
und QUIC / TLS 1.3 (rechts)**

5.4 Internet Control Message Protocol

Grundlagen

- **ICMP** ist ein erweiterbares Protokoll-Framework für Test-, Diagnose- und Fehlermeldedefunktionen in und für IP
 - ICMPv4 (RFC 792) Pakete werden in IPv4 Paketen enkapsuliert
 - ICMPv6 (RFC 4443) Pakete werden in IPv4 Paketen enkapsuliert
- ICMP gilt offiziell als Schicht 3 Protokoll, obwohl es...
 - ... End-to-end Funktionalität aufweist
 - ... von IP Paketen enkapsuliert wird



ICMP Enkapsulierung

5.4 Internet Control Message Protocol

Struktur eines ICMP Pakets

0 - 3	4 - 7	8 - 11	12 - 15	16 - 19	20 - 23	24 - 27	28 - 31
Type		Code		Checksum			
Rest of Header (je nach Typ und Code)							
Data (Optional, max. 65535-28 Bytes)							

- Type: Typ der ICMP Nachricht (maximal $2^8 = 256$ verschiedene Typen möglich)
- Code: Subtyp der ICMP Nachricht (spezifiziert den Typ genauer, maximal $2^8 = 256$ verschiedene möglich)
- Checksum: Prüfsumme für die gesamte ICMP Nachricht (also ohne IP Header)
- Rest of Header: Der Inhalt des Restheaderfelds hängt von Type und Code der Nachricht ab

5.4 Internet Control Message Protocol

ICMP Message Types

Type	Name	Code	Beschreibung
0	Echo Reply	0	Antwort auf Echo Request
3	Destination Unreachable	0	Destination network unreachable
		1	Destination host unreachable
		3	Destination port unreachable
8	Echo Request	0	Antwort durch Echo Reply
11	Time Exceeded	0	TTL expired in transit
		1	Fragment reassembly time exceeded

5.4 Internet Control Message Protocol

Struktur eines Ping Requests

0 - 3	4 - 7	8 - 11	12 - 15	16 - 19	20 - 23	24 - 27	28 - 31
Type		Code		Checksum			
Identifier				Sequence Number			
Data (Optional, max. 65535-28 Bytes)							

- Type: Immer 8 (Echo Request); Code: Immer 0 (nicht benötigt)
- **Identifier**: Identifiziert eine "Ping Session"
- Sequence Number: Identifiziert eine Echo Request Nachricht in einer "Ping Session"
- Data: Data sind in der Regel Characters von a bis w durchlaufend und wieder neu beginnend